Algoritmos Distribuidos

Cálculo de Estado Global

Andrés Sanoja
Claudia León

Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación

Octubre 2008 –Febrero 2022



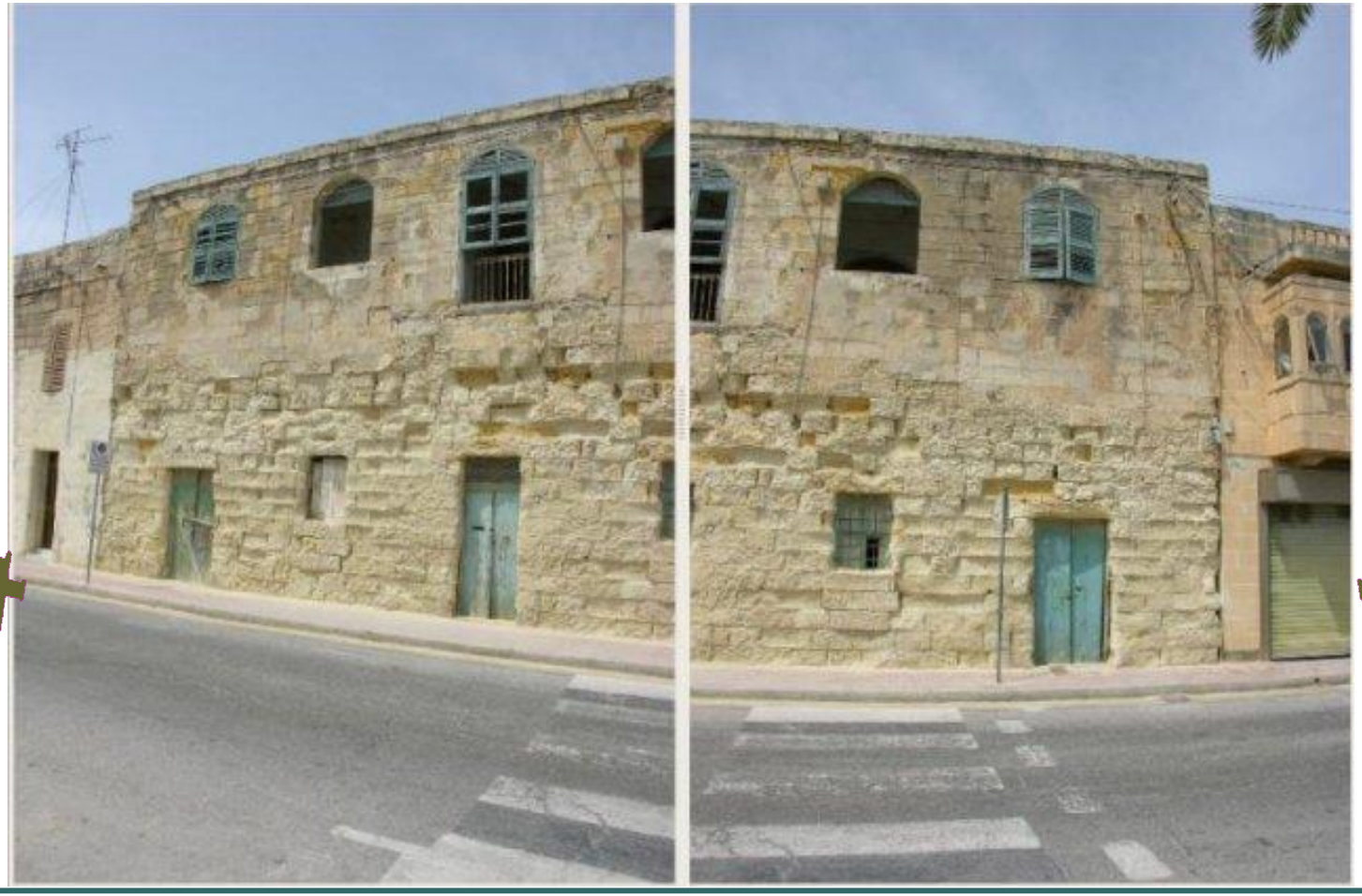
Objetivos

- Comprender y describir el problema de determinación de un estado global
- Comparar diferentes soluciones para el cálculo de estados globales.

Ejemplo: Fotografía Panorámica

- Dos fotógrafos quieren tomar juntos la foto de una edificación
- Uno toma la parte izquierda y el otro la parte derecha
- Luego deben unir las dos fotos y obtener la conjunta

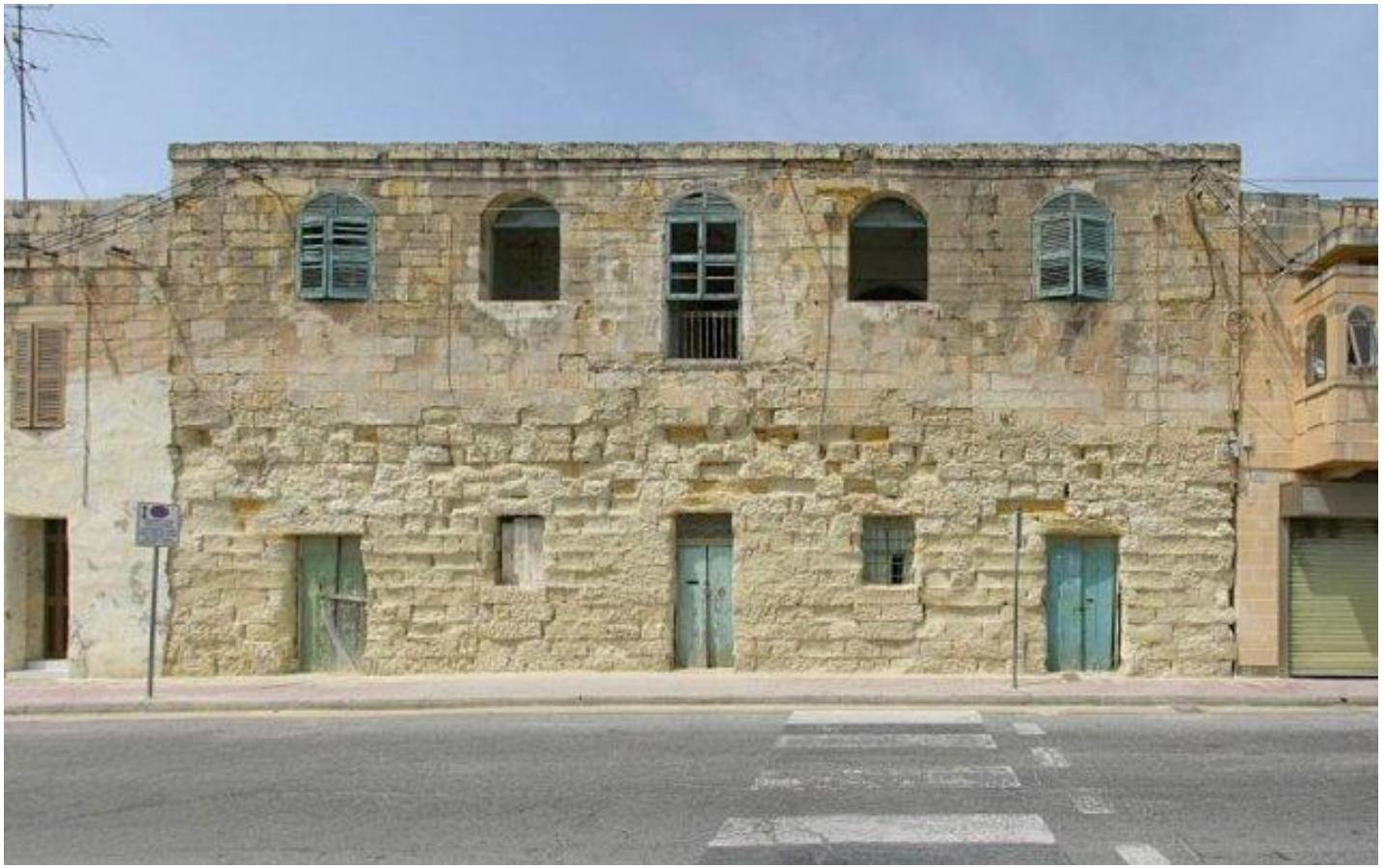
Ejemplo: Fotografía Panorámica



Ejemplo: Fotografía Panorámica

- Aun cuando la diferencia es mínima, cada foto fue tomada en diferente momento
- Cada foto tiene un ángulo distinto
- ¿Una Solución?: Tratar de trabajar en conjunto para enfocar las cámaras y lo mas coordinadamente posible

Lo Esperado



Pero Generalmente se tiene…



- Cada foto tiene una perspectiva y tiempo diferente, pero se aprecia lo observado (la montaña) como un todo

El problema de Cálculo de un Estado Global

Cuando se tiene **el control distribuido**, se hace necesario que los nodos encargados de asegurar el control conozcan el estado global del sistema.

El control distribuido presenta las siguientes características :

i) En un instante dado, un nodo sólo tiene un conocimiento aproximado del estado de los otros nodos.

ii) Los eventos que ocurren en un orden dado en un nodo, pueden ser percibidos de manera diferente en los otros.

El problema de Cálculo de un Estado Global

Objetivo: Disponer de un algoritmo que permita a uno o más nodos obtener un *estado global coherente* es fundamental en un sistema distribuido.

Un proceso que desee obtener un **estado global** debe obtener **el estado de c/u de los otros procesos y el estado de c/u de las líneas de comunicación**, siendo importante el orden en que obtiene dichos estados.

Estos estados están relacionados unos con otros y su unión debe proveer un **estado global coherente**.

Cálculo de un Estado Global: Ejemplo 1

Consideremos un A.D. que consiste de dos procesos p y q, conectados a través de una línea bidireccional c

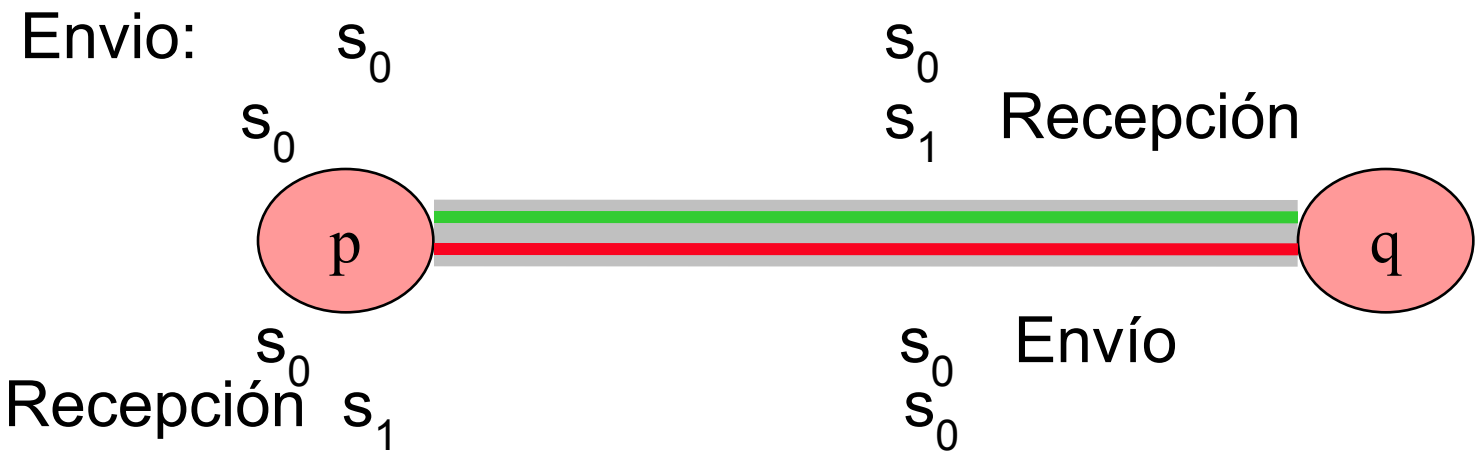


La aplicación consiste de un *token* (marcador) que es pasado de un proceso al otro.

Cada proceso tiene dos *estados*:

- S_0 es el *estado* en el cual el proceso no posee el *token*
- S_1 es el *estado* en el cual el proceso posee el *token*

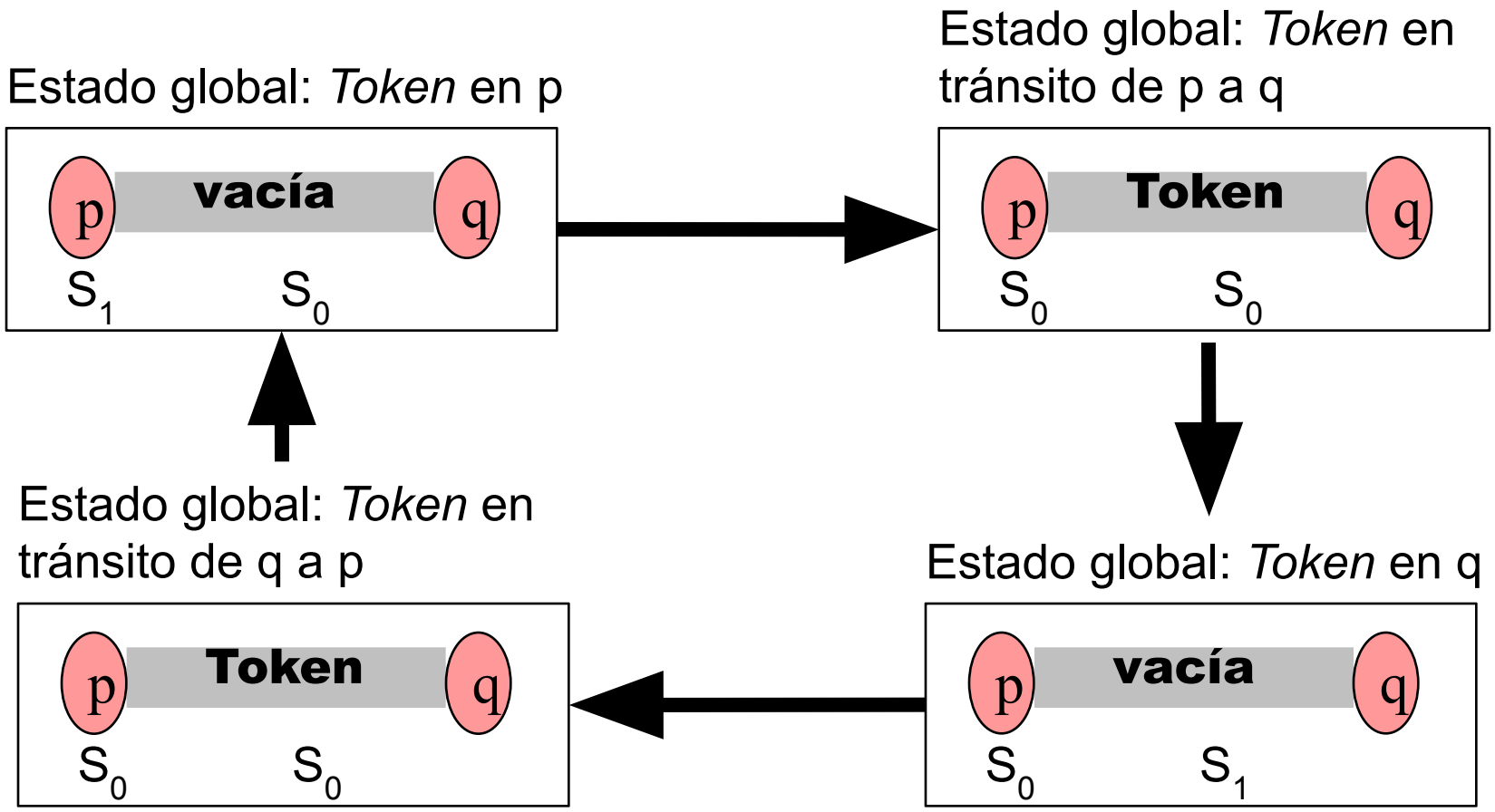
Cálculo de un Estado Global: Ejemplo 1



Cada proceso tiene dos *eventos*:

- Una transición de s_1 a s_o con el envío del *token*
- Una transición de s_0 a s_1 con la recepción del *token*

Estados globales y transiciones: Ejemplo 1



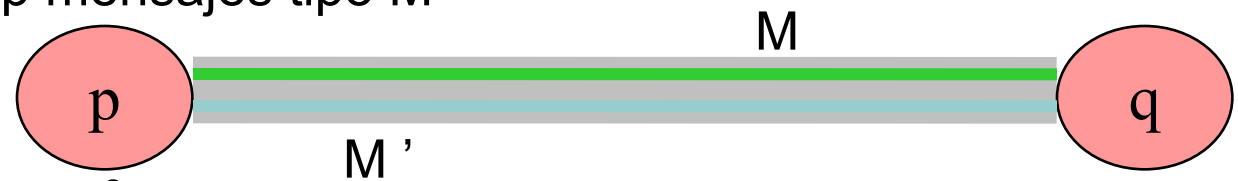
Dificultades para el cálculo de un estado global

Un algoritmo de Cálculo de un Estado Global **no es** trivial debido a que:

1. El objeto que se observa puede modificarse durante la observación.
2. Es necesario que la observación no altere el objeto.

Cálculo de un Estado Global: Ejemplo 2

Consideremos una A.D. que consiste de dos procesos p y q, conectados a través de una línea bidireccional. p envía a q mensajes tipo M y q envía a p mensajes tipo M'



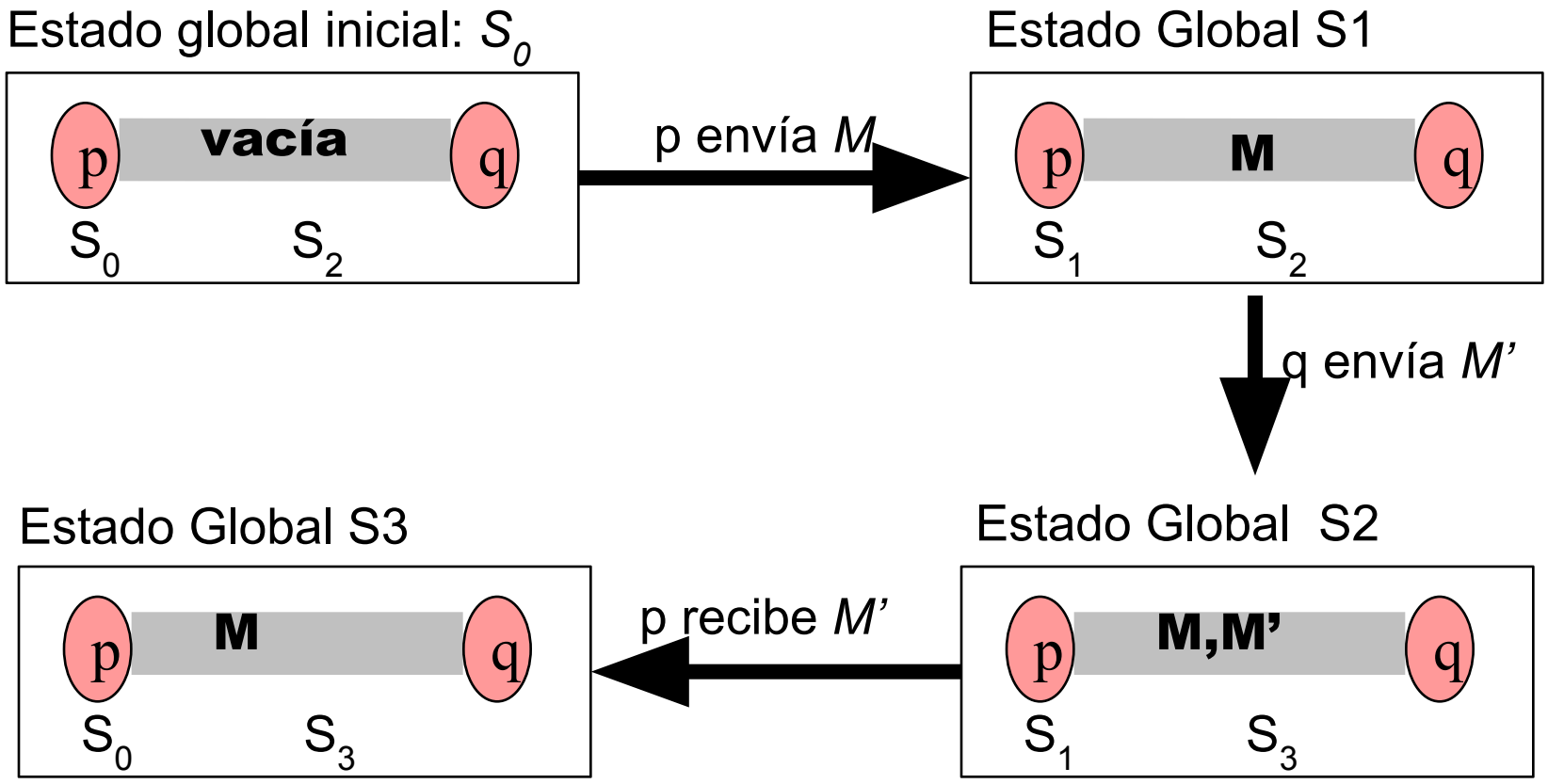
Inicial: s0 s2

S': s1 s3

Cada proceso tiene dos *estados*

- s_0 es el *estado* en el cual p puede enviar mensajes tipo M
- s_1 es el *estado* en el cual p puede recibir mensajes tipo M'
- s_2 es el *estado* en el cual q puede enviar mensajes tipo M'
- s_3 es el estado en el cual q puede recibir mensajes tipo M

Estados globales y transiciones: Ejemplo 2



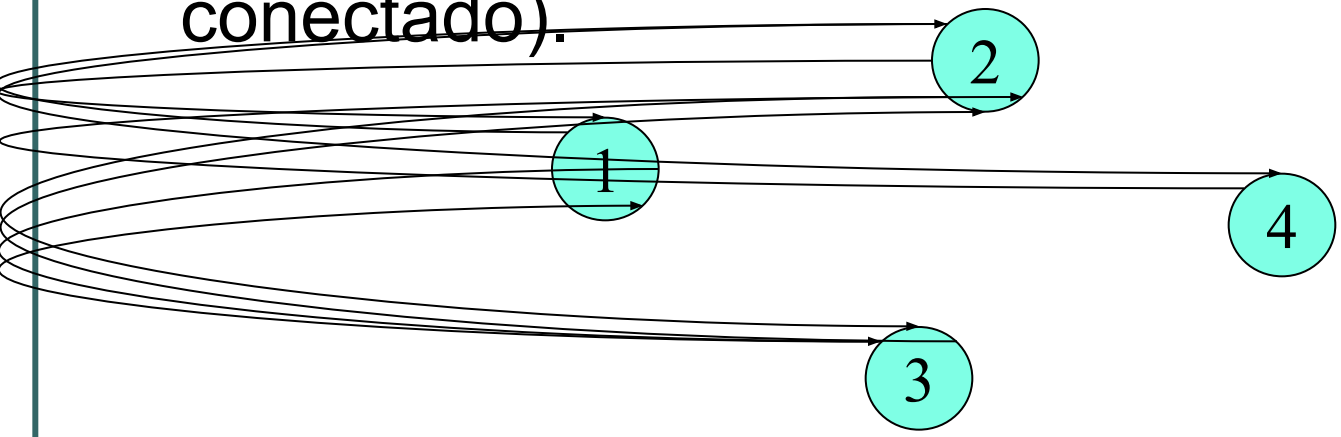
Objetivo de un algoritmo de cálculo de estado global

Un algoritmo de cálculo de estado global tiene como objetivo construir explícitamente una configuración del sistema compuesta por los estados locales de todos los proceso, donde:

1. El registro del estado local debe ser iniciado en cada proceso
2. En el estado global, no existen mensajes *posteriores* al registro del estado local por el emisor, que sean recibidos *antes* del registro del estado local del receptor.

Definiendo Estados Globales

No necesariamente todos los procesos deben estar interconectados, pero cada proceso debe tener, al menos, un canal de entrada y uno de salida y debe ser posible para cada proceso alcanzar otro proceso (grafo fuertemente conectado).



Definiciones

- **Estado Local:** Para un proceso P_i, su estado local LSP_i , en un momento dado está definido por el contexto local de la aplicación distribuida.
- **send(m_{ij}):** denota el evento de envío del mensaje m_{ij} por el proceso P_i al Proceso P_j.
- **receive(m_{ij}):** denota el evento de recepción del mensaje m_{ij} por el proceso P_j.
- **time(LSP_i):** denota el momento en el cual LSP_i fué registrado.
- **time(send(m_{ij})):** denota el tiempo en el cual ocurrió send(m_{ij}).

Definiciones

- Para un mensaje m_{ij} enviado por P_i a P_j, decimos que:
  - $send(m_{ij}) \in LSP_i \Leftrightarrow time(send(m_{ij})) < time(LSP_i)$
  - $rec(m_{ij}) \in LSP_j \Leftrightarrow time(rec(m_{ij})) < time(LSP_j)$
- Para los estados locales LSP_i y LSP_j se definen dos conjuntos que contienen mensajes enviados por P_i a P_j:
  - $Transito(LSP_i,LSP_j) = \{m_{ij} / send(m_{ij}) \in LSP_i \wedge rec(m_{ij}) \notin LSP_j\}$
  - Inconsistente(LSPi,LSPj) =

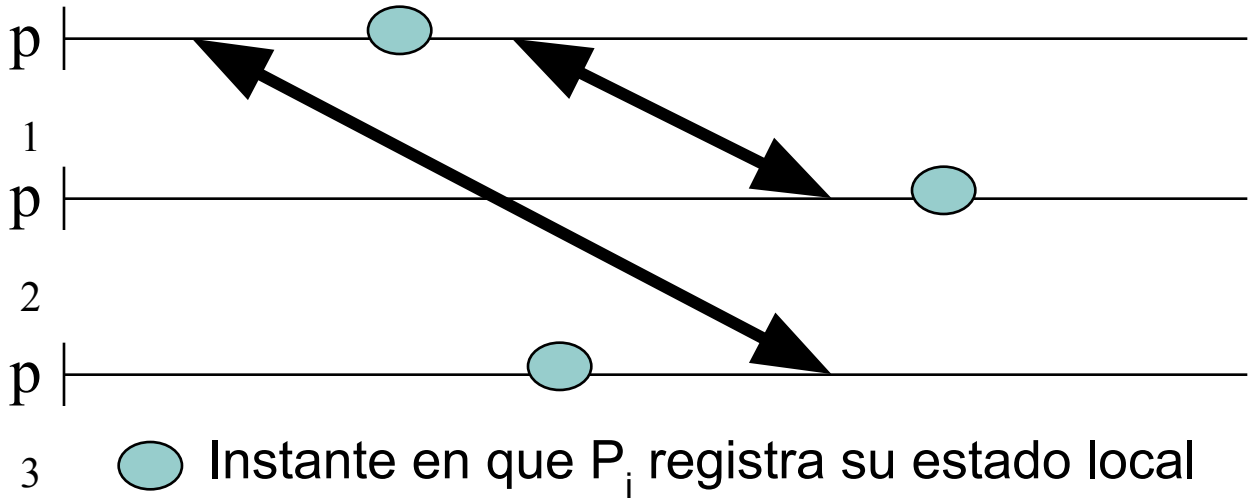
$$\{m_{ij} / send(m_{ij}) \notin LSP_i \wedge rec(m_{ij}) \in LSP_j\}$$

Estado Global Coherente (o Consistente)

- **Estado Global:** Un estado global GS de una aplicación distribuida es una colección de los estados locales de los procesos que la conforman. GS={LSP_1,LSP_2, … …,LSP_n} donde *n* es la cantidad de procesos.
- **NOTA**: no toda colección de estados locales representa une **estado global coherente**.

Un estado global GS={LSP_1,LSP_2, … …,LSP_n} **es coherente** $\Leftrightarrow$ $\forall$ i,j : inconsistente(LSP_i,LSP_j) = $\varnothing$

Ejemplo de un estado global incoherente



Instante en que P_i registra su estado local

Estado local de p_1: {Un mensaje enviado de p_1 a p_3}

Estado local de p_2: {Un mensaje recibido de p_1}

Este es un mensaje enviado en un evento a posteriori

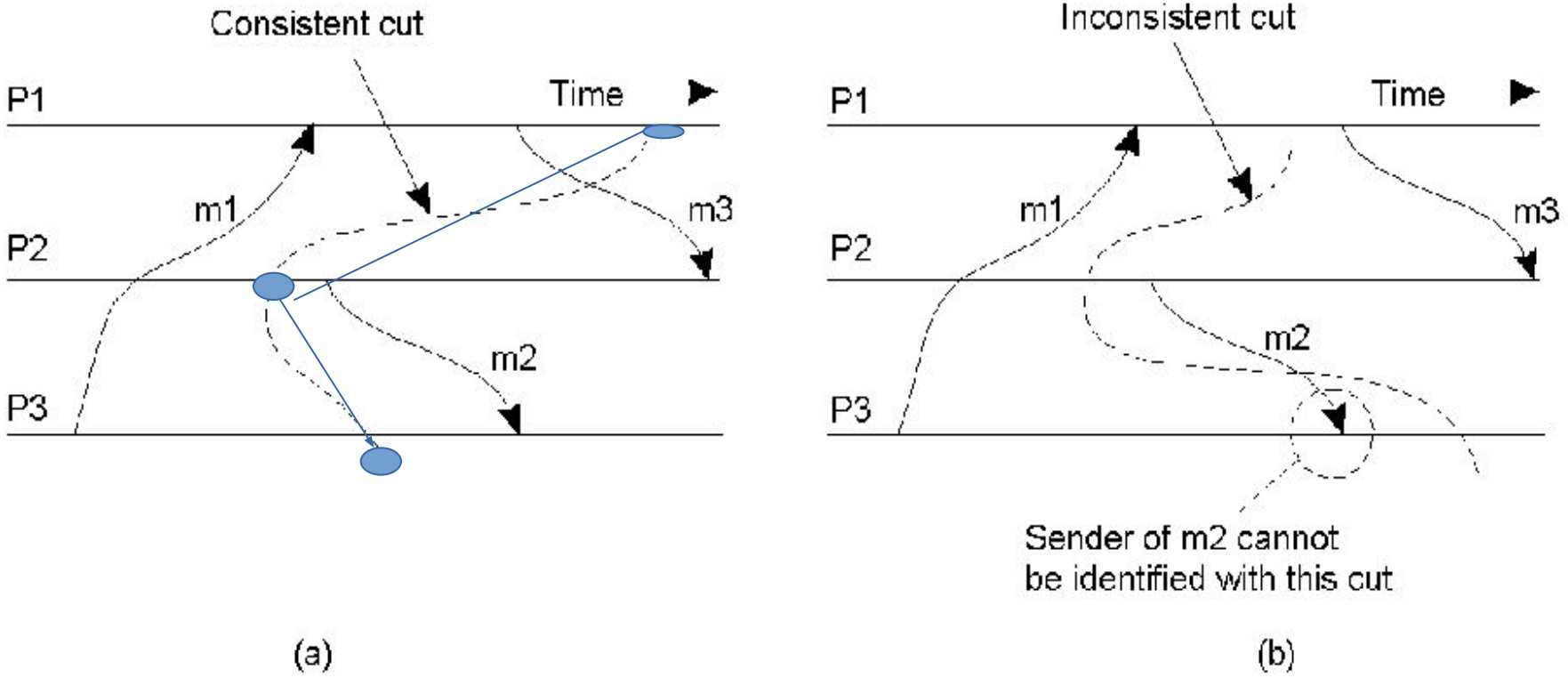
Estado Local de p_3: {No hay mensajes recibidos de p_1}

OK el mensaje está en tránsito

Corte Consistente

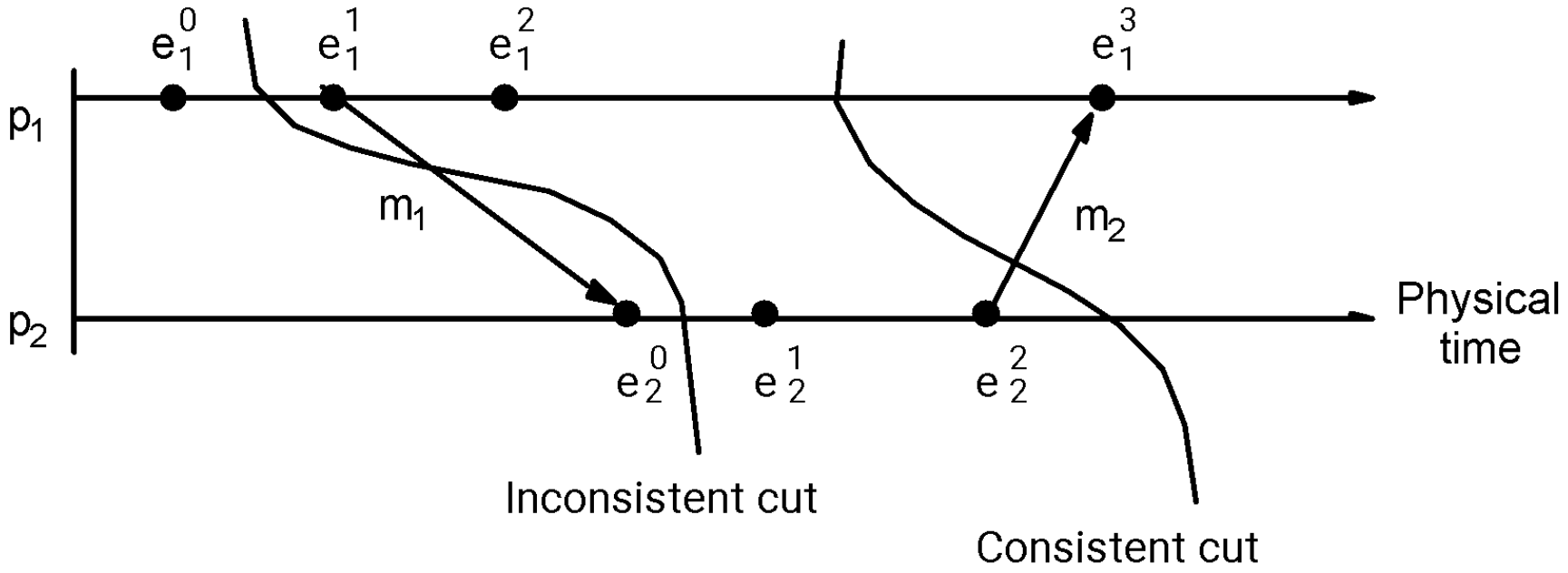
- Decimos que en *EG* debe presentar un corte consistente
- Una manera de definir un corte consistente es que los resultados de los estados LSP_i deberían ocurrir concurrentemente
- También, es cuando todos los eventos que ocurren **antes** del corte preceden a aquellos que ocurren **después** del corte

Estados Globales - Cortes



- Corte consistente
- Corte inconsistente

Estados Globales - Cortes



Cálculo de estado global

Importante: Los algoritmos de cálculo de estado global (o *snapshots*) *no producen* el estado que *ocurre actualmente en el sistema*, sino uno entre los estado posibles que *pudieron haber ocurrido*.

Utilidad de algoritmos de cálculo de un estado global

1. Detección de propiedades (estables como *deadlock*, terminación, etc) en una aplicación distribuida.
2. Recuperación en caso de fallas: en lugar de restaurar el estado inicial, se restaura un estado global consistente previamente calculado.
- Como herramienta de *debugging* de aplicaciones distribuidas.

1. Recolector de Basura

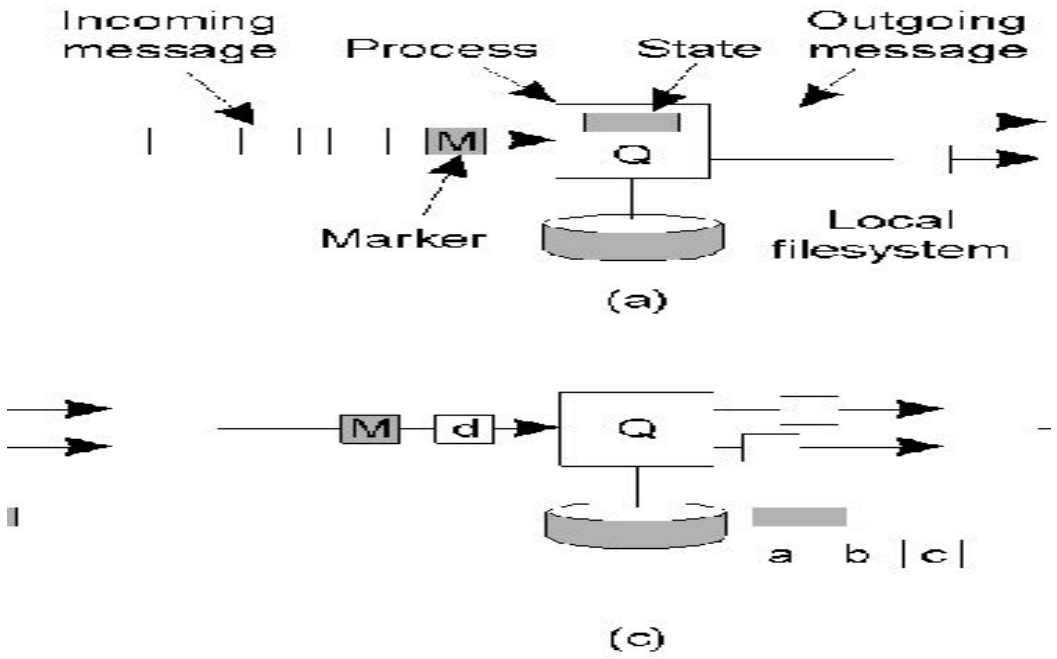
Algoritmo de Chandy y Lamport

1985

Hipótesis

- La comunicación es fiable
- La comunicación es F.I.F.O. :
  NO hay pérdida de secuencia

Algoritmo de Chandy y Lamport



a) Organización de un proceso y sus canales para el cálculo de un estado global

Algoritmo de Chandy y Lamport

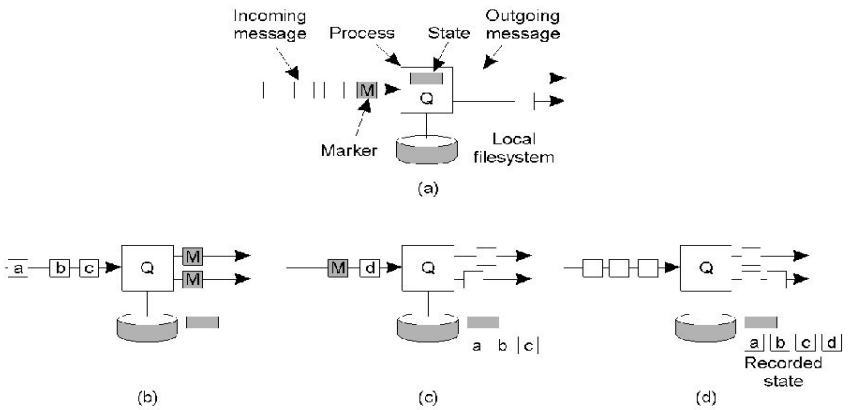
En el algoritmo de Chandy y Lamport [1985] los procesos se informan unos a otros que deben registrar su estado local, a través del envío de un mensaje especial de control, llamado *marcador, de acuerdo a las siguientes reglas:*

- Cuando un proceso desea iniciar el cálculo de un estado global, registra su estado y envía el mensaje *marcador* a todos sus vecinos
- Cuando un proceso que no ha registrado su estado local, recibe un mensaje marcador, entonces registra su estado local, marca como vacío los mensajes en transito desde el emisor, envía el marcador a sus vecinos ANTES de enviar cualquier otro mensaje de la aplicación.

Algoritmo de Chandy y Lamport

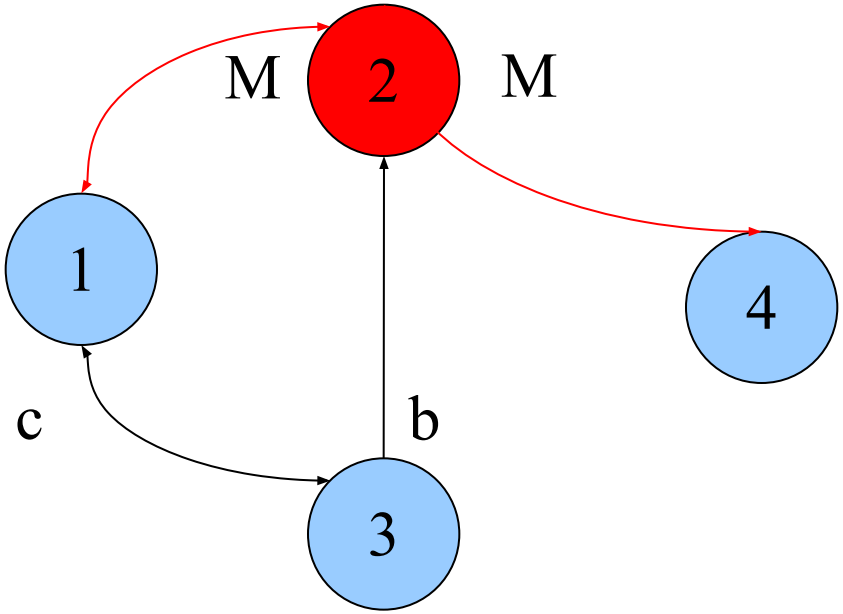
- Cuando un proceso que ya registró su estado recibe un mensaje de aplicación de otro proceso del cual no ha recibido el marcador, guarda el mensaje como “en transito desde el emisor”.
- Cuando un proceso que ya registró su estado local recibe un mensaje marcador, cierra el conjunto de mensajes “en transito desde el emisor”.
- El estado global no es construido, es necesario recolectarlo. Una manera es agregar el ID del iniciador al marcador, y al final todos los procesos le envían su estado al iniciador.

Algoritmo de Chandy y Lamport



- El proceso Q recibe una marca por primera vez y almacena su estado local
- Q almacena todos los mensajes entrantes
- Q recibe una marca de su canal de entrada y finaliza el almacenamiento de los mensajes desde el canal de entrada

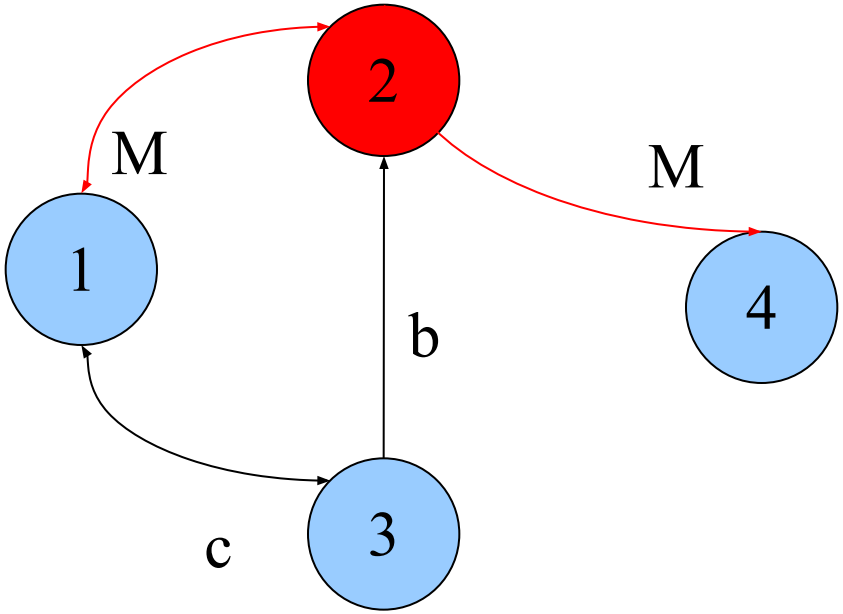
Estado Global (Snapshot)



Registrado:
Estado LSP2

Nodo 2 inicia el “snapshot”

Estado Global (Snapshot)

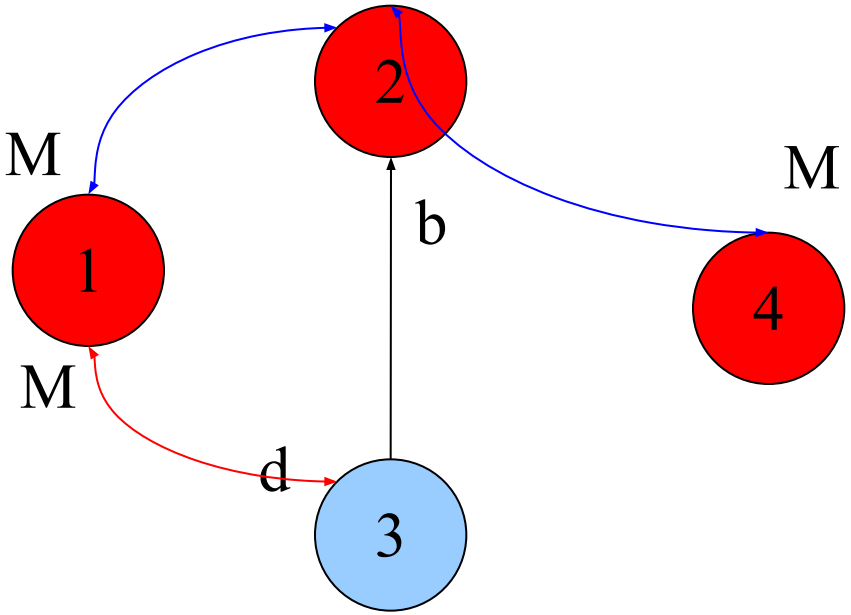


Registrado:

Estado LSP2

Nodo 2 inicia el “snapshot”

Estado Global (Snapshot)



Registrado:

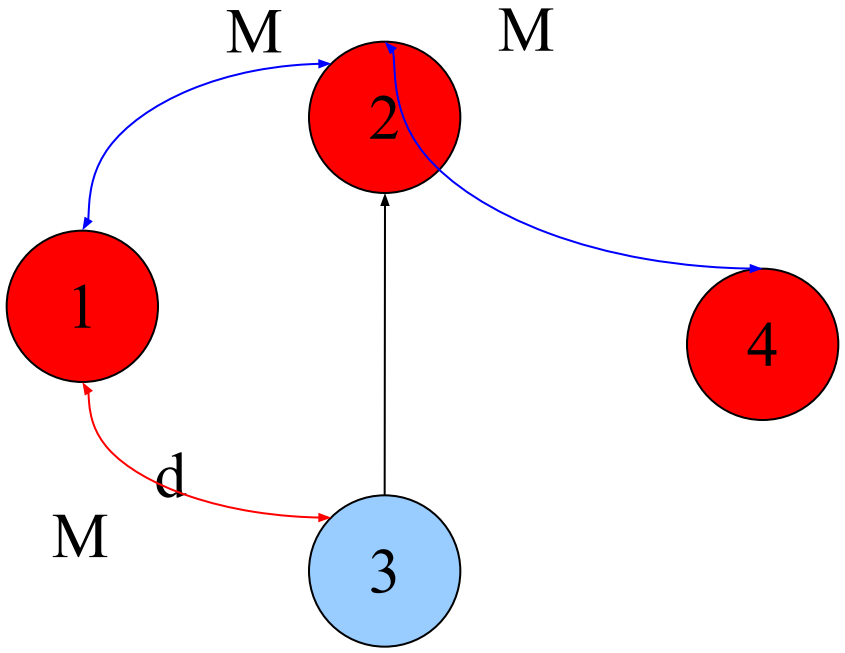
Estado LSP2

Estado LSP1

Estado LSP4

Nodo 2 inicia el “snapshot”

Estado Global (Snapshot)



Registrado:

Estado LSP2

Estado LSP1

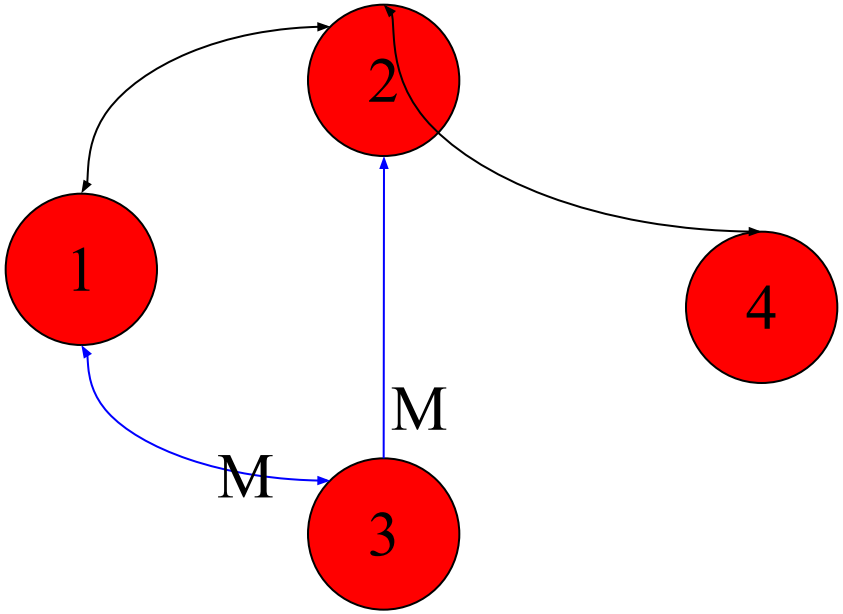
Estado LSP4

Transito(3,2)=b

Transito(1,2)=vacio

Transito(4,2)=vacio

Estado Global (Snapshot)



Registrado:

Estado LSP2

Estado LSP1

Estado LSP4

Transito(3,2)=b

Transito(1,2)=vacio

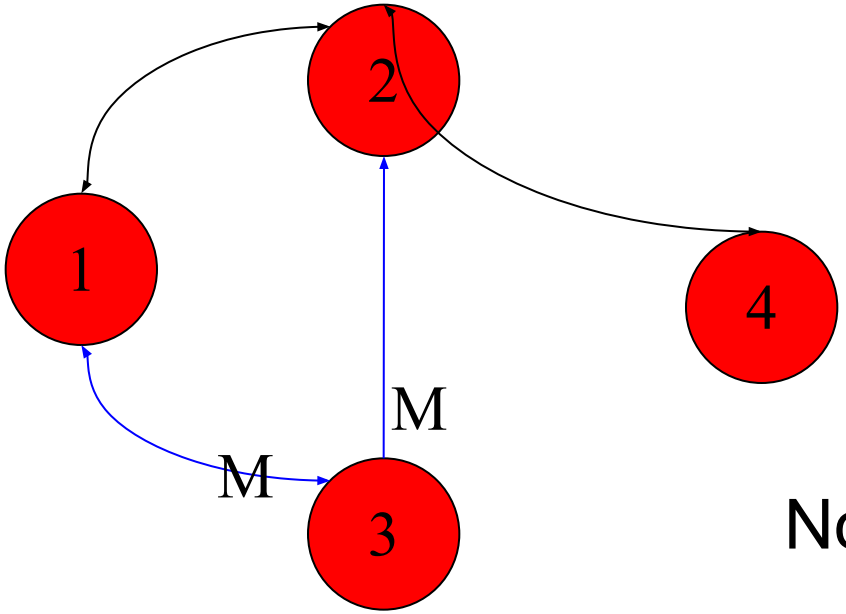
Transito(4,2)=vacio

Estado LSP3

Transito(4,2) = vacio

Transito(3,1) = d

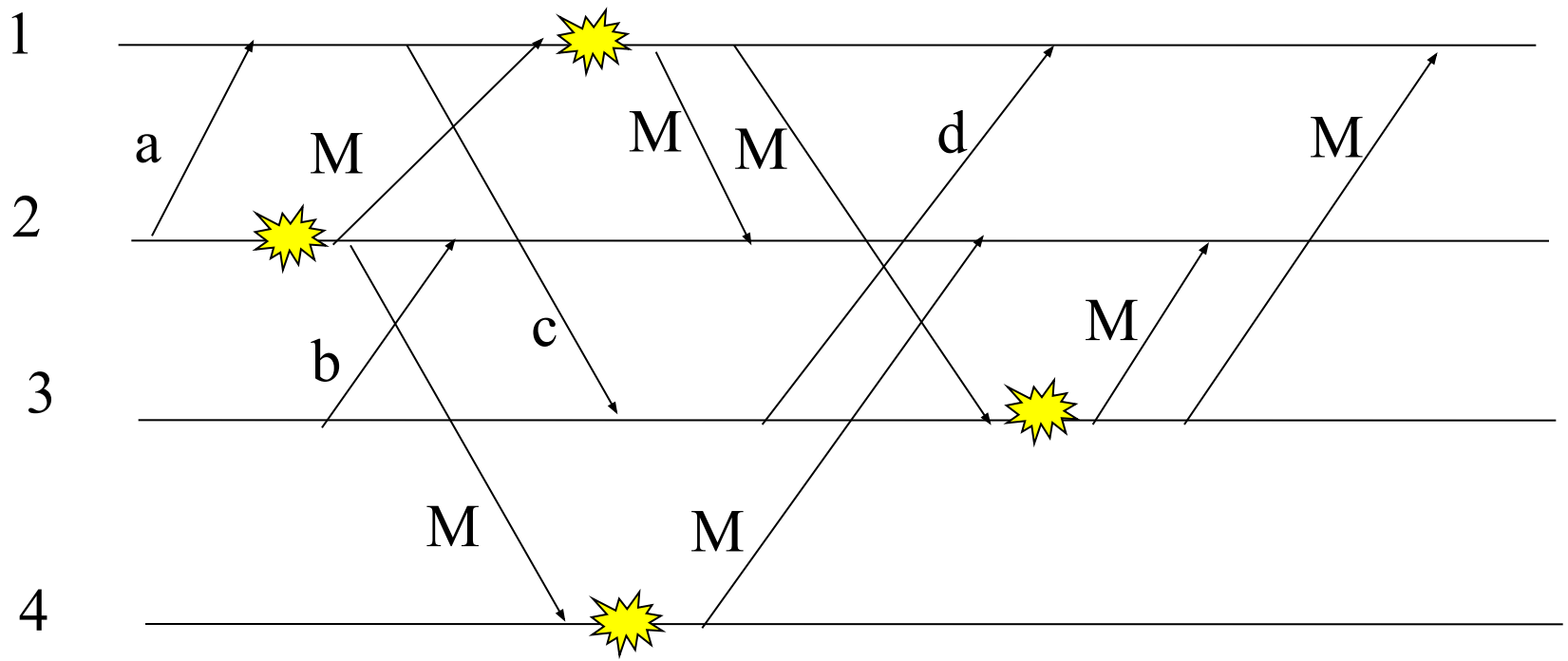
Chandy Lamport Snapshot



Utiliza O(|E|) mensajes donde E es el numero de nodos del grafo y |E| la cantidad de arcos

Nota: por ejemplo, entre 1 y 2 hay dos arcos (uno de entrada y uno de salida) y entre 3 y 2: sale de 3 y entra en 2.

Chandy Lamport Genera un Corte Consistente



Algoritmo de Chandy y Lamport

Mensaje de control: marcador; % mensaje de registro de estado local
Mensaje de aplicación: (Aplicación,Men) %Mensaje de la aplicación

Var marca : lógico inicializa falso; %indica si ya registró o no su estado
FinTransito : Arreglo[Vecinos] de lógico inicaliza falso;
Transito : Arreglo[Vecinos] de Conjunto de Mensajes inicializa vacío;
%Almacena los mensajes en transito de cada vecino

% Para iniciar el algoritmo.

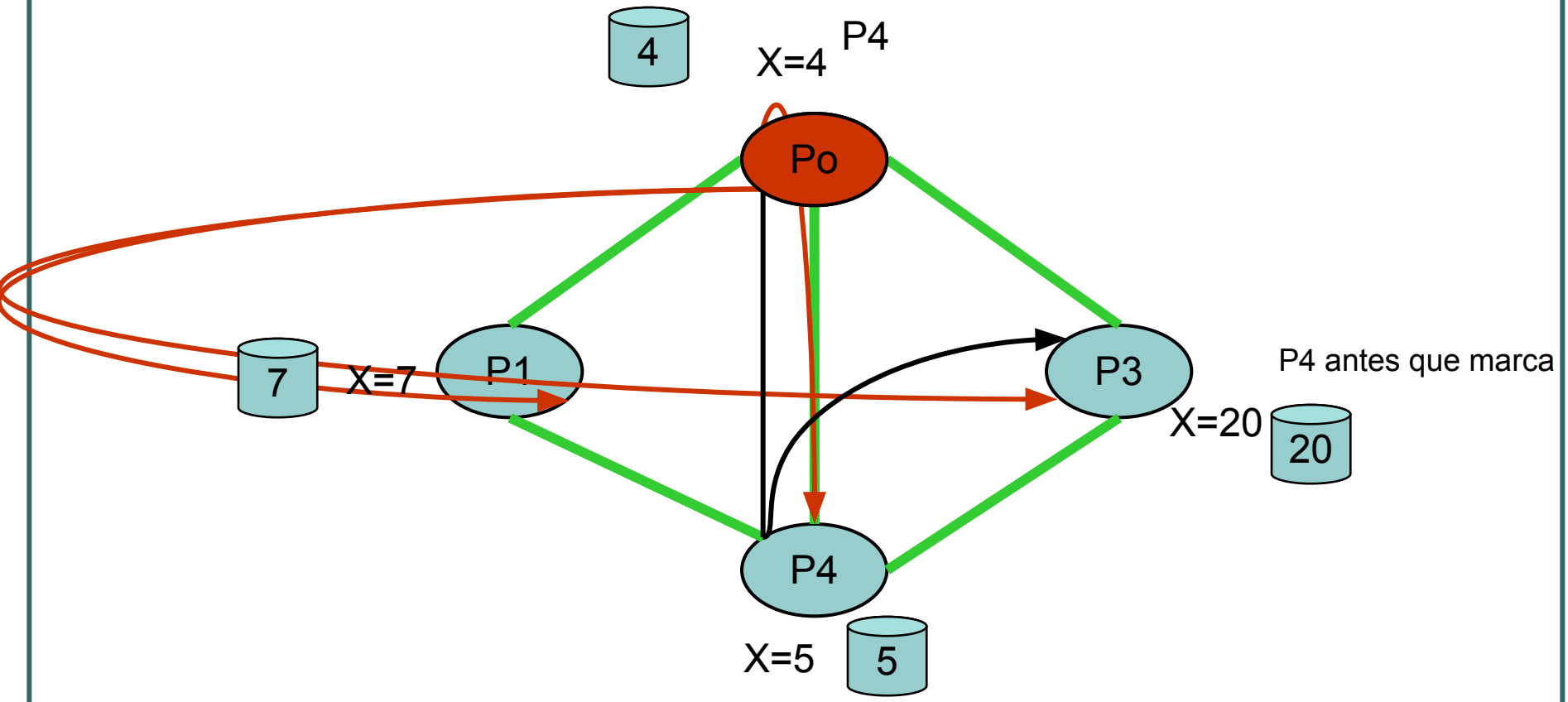
```
Registrar-estado-local;
marca:= verdad;
paratodo j ∈ Vecinos hacer
    Enviar(marcador) a j;
fparatodo;
```

Algoritmo de Chandy y Lamport

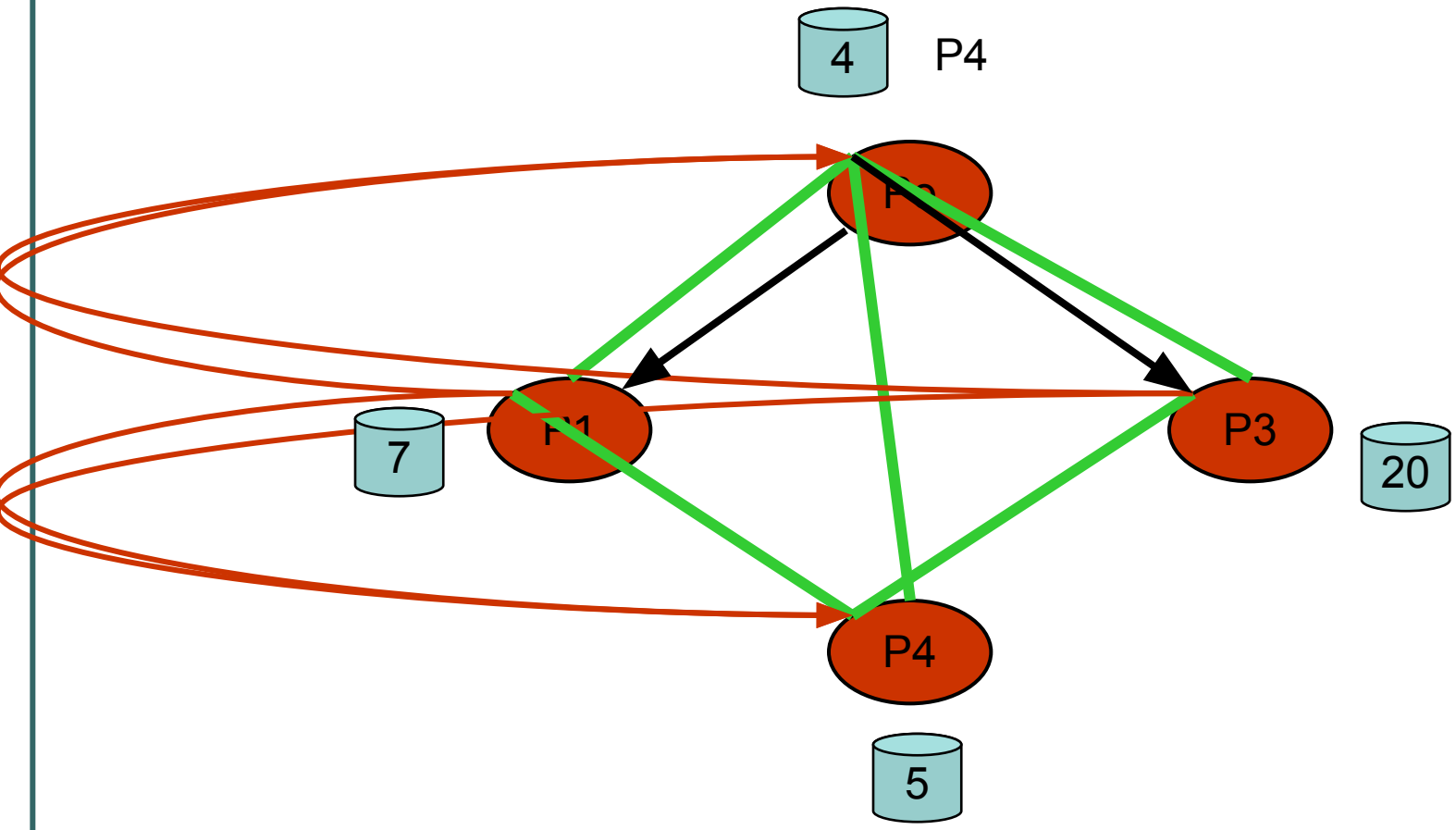
```
Cuando {llegue un mensaje de tipo marcador} hacer
        Recibir (marcador) de emisor;
        FinTransito[emisor]:=verdad;
        Si not marca entonces
              marca:= verdad;
        Registrar-estado-local;
              paratodo j∈ Vecinos hacer Enviar(marcador) a j;  fparatodo;
        fsi;
  fincuando;

Cuando {llegue un mensaje de tipo aplicación} hacer
        Recibir (aplicación, Men) de emisor;
        Si marca and not Fintransito[emisor] entonces
Insertar(Men,Transito[emisor]);
        fsi;
fincuando;
```

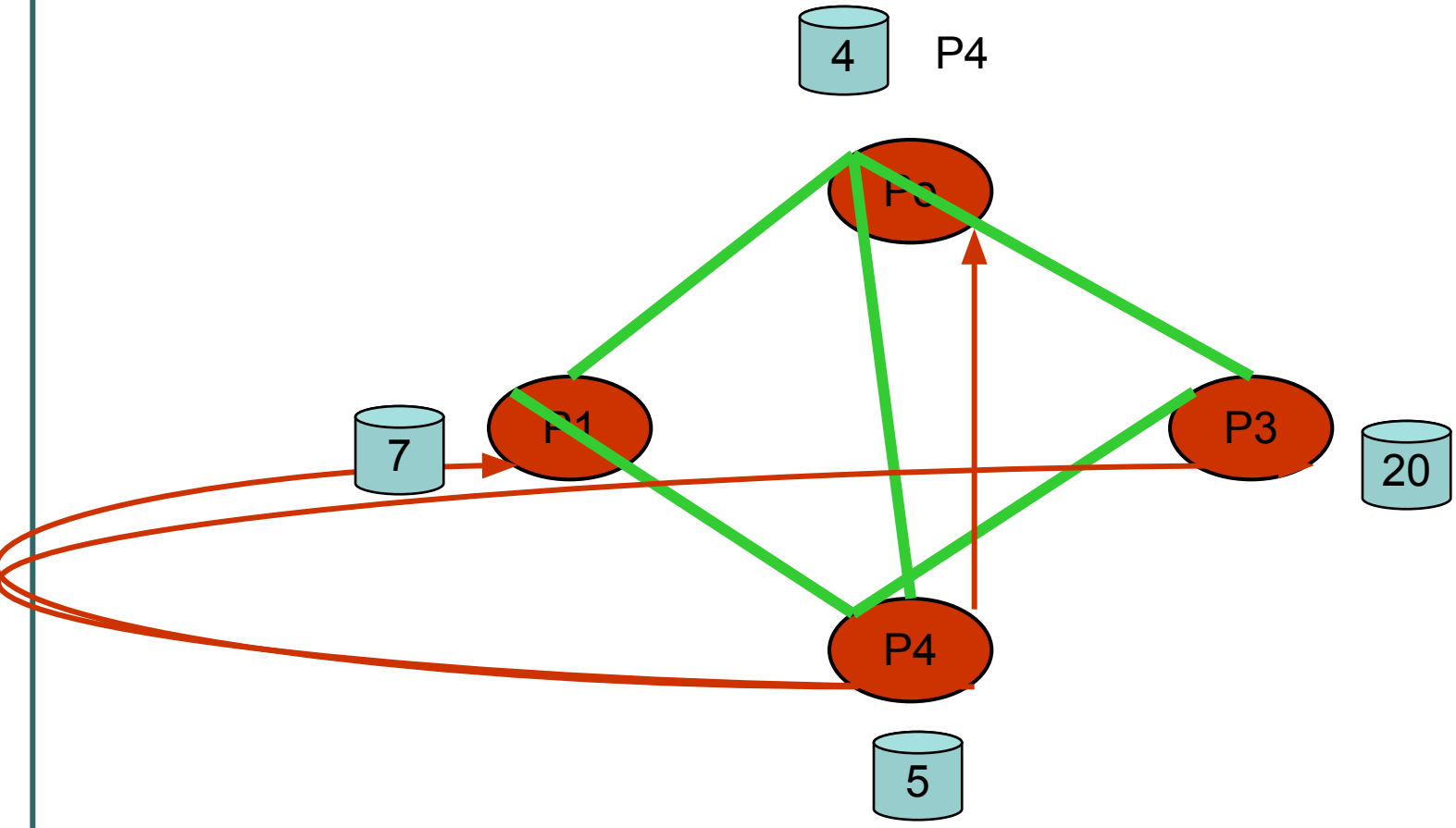
Algoritmo de Chandy y Lamport: Ejemplo



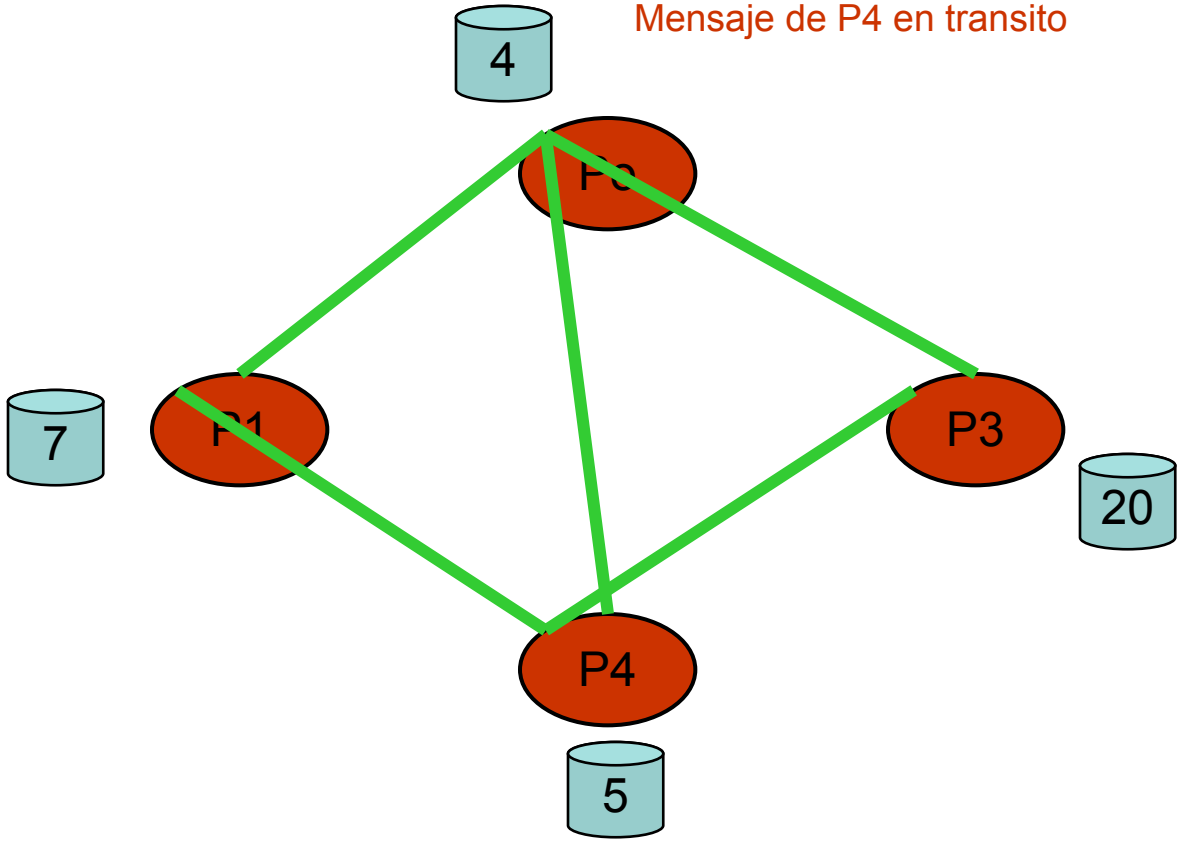
Algoritmo de Chandy y Lamport: Ejemplo



Algoritmo de Chandy y Lamport: Ejemplo



Algoritmo de Chandy y Lamport: Estado Global Calculado



Algoritmo de Chandy y Lamport:

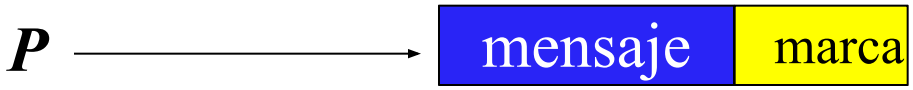
- Los marcadores conjuntamente con las líneas FIFO actúan como delimitadores de los mensajes sobre las líneas.
- El marcador diferencia entre los mensajes que deben ser registrados en el estado global que se está calculando y aquellos que no deben ser registrados.
- El cálculo del estado global puede ser iniciado por varios procesos concurrentemente, teniendo cada proceso su propia "versión" de estado global, agregando el Id del proceso al marcador.
- La condición FIFO no se puede garantizar en muchos sistemas distribuidos.

Algoritmo de Lai y Yang : Características

- No cuenta con la propiedad FIFO de los canales.
- No es posible separar los mensajes preshot y los postshot por marcadores. Se utilizan sólo los mensajes propios de la aplicación.
- Cada mensaje de aplicación es etiquetado con
- Información relevante sobre si es un postshot o preshot.
- Es un algoritmo para guardar el estado global en cada proceso, mas no los recolecta.

Algoritmo de Lai y Yang : Características

- Cuando un proceso *p* envía un mensaje, le concatena el valor de la marca.



- El algoritmo inspecciona el mensaje entrante y graba su estado local antes de recibir el primer mensaje de *postshot*.

Algoritmo de Lai y Yang : Notación

Marca: indica si el mensaje es un ***preshot*** o ***postshot.***

Marca= *verdad,* es un postshot.

Marca= *falso,* es un preshot.

Y se denota: **(mens,marca)**

Algoritmo de Lai y Yang

```
Var Marca: logico  inicializado en falso;

% Para iniciar el algoritmo:
   grabar el estado local; Marca:= verdad;

%Para enviar un mensaje:
       enviar(mens,Marca)

Cuando (Llega un mensaje) (mes,c):
       recibir(mens,c);
       Si (c and not Marca) Entonces
      grabar el estado local;  Marca:=verdad
    Fsi;
Fcuando
```

Algoritmo de Lai y Yang: Ventajas

- No se envían mensajes de control
- Se incluye explícitamente en el mensaje una marca que indica si es un *preshot* o un *postshot,*
- Si una comunicación eventual con cada proceso es garantizada, siempre tendrá lugar un *snapshot* completo
- Como no se maneja la condición FIFO en el canal un proceso *q* puede recibir *preshot* y *postshot de forma alterna*
- Conveniente en sistemas donde exista GIM (Gran Intercambio de Mensajes)

Algoritmo de Lai y Yang: Desventajas

- El hecho de que no se envíen mensajes de control no asegura que todos los procesos guarden su estado
- En sistemas donde no exista una alta comunicación entre los procesos, la probabilidad de que ocurra un *snapshot* completo es menor.

